Containerlab

HPE Aruba AOS-CX VSX & Juniper vJunos-switch MC-LAG

Network Virtualization Lab Guide

Nicolas Culetto — Channel Presales, HPE Aruba Networking

github.com/Luconik/netdevops | Avril 2026 / April 2026

Version AOS-CX	AOS-CX 10.16.1020
Version vJunos-switch	25.4R1.12
Containerlab	0.74.3
Plateforme / Platform	Fedora 43 bare metal (i9-10850K / 128GB RAM)
Repo GitHub	github.com/Luconik/netdevops
Guide compte HPE / HPE account guide	github.com/Luconik/hpe-aruba-guides/tree/main/nsp-account

Sommaire / Table of Contents

1. Introduction

Objectif / Goal

Ce guide décrit la mise en place de deux labs réseau virtualisés avec **Containerlab** — l'un avec des switches **HPE Aruba AOS-CX** en configuration VSX (Virtual Switching Extension), l'autre avec des switches **Juniper vJunos-switch** en configuration MC-LAG. Ces labs permettent de pratiquer les technologies d'agrégation multi-châssis des deux gammes HPE Networking sans matériel physique.

This guide describes the setup of two virtualized network labs using Containerlab — one with HPE Aruba AOS-CX switches in VSX configuration, the other with Juniper vJunos-switch in MC-LAG configuration. These labs enable practicing multi-chassis aggregation technologies from both HPE Networking product lines without physical hardware.

Containerlab

Containerlab est un outil open-source qui déploie des topologies réseau virtualisées à partir d'un simple fichier YAML (*topology.yml*). Il supporte les images Docker natives et les VMs packagées via **vrnetlab** (QEMU dans Docker). Le plugin VS Code Containerlab offre une visualisation graphique de la topologie et un accès SSH direct par clic droit sur les nodes, compatible avec VS Code Remote SSH.

2. Prérequis / Prerequisites

Infrastructure

Composant	Valeur / Value
OS	Linux bare metal — Fedora 43 / Ubuntu 24.04
CPU	x86_64 avec virtualisation KVM
RAM	16 GB minimum (vJunos ~5 GB par switch)
Containerlab	≥ 0.74.3 — containerlab.dev/install
Docker	Version récente / Recent version
VS Code	Extension Containerlab by srl-labs (optionnel)

Images requises / Required Images

Image Docker	Source	Compte / Account
vrnetlab/vr-aoscx:10.16.1020	networking.hpe.com — AOS-CX Switch Simulator	Oui / Yes *
vrnetlab/juniper_vjunos-switch:25.4R1.12	juniper.net/us/en/dm/vjunos-labs.html	Non / No

** Guide création compte HPE / HPE account creation guide : github.com/Luconik/hpe-aruba-guides/tree/main/nsp-account*

Build des images vrnetlab / Building Images

AOS-CX :

```
tar xf AOS-CX_10_16_1020.ova
cp AOS-CX_10_16_1020.vmdk vrnetlab/aruba/
cd vrnetlab/aruba && make
```

vJunos-switch :

```
cp vJunos-switch-25.4R1.12.qcow2 vrnetlab/juniper/vjunosswitch/
cd vrnetlab/juniper/vjunosswitch && make
# Build ~10 min — image finale ~9 GB / Final image ~9 GB
```

3. Lab AOS-CX VSX — HPE Aruba

Liens / Links

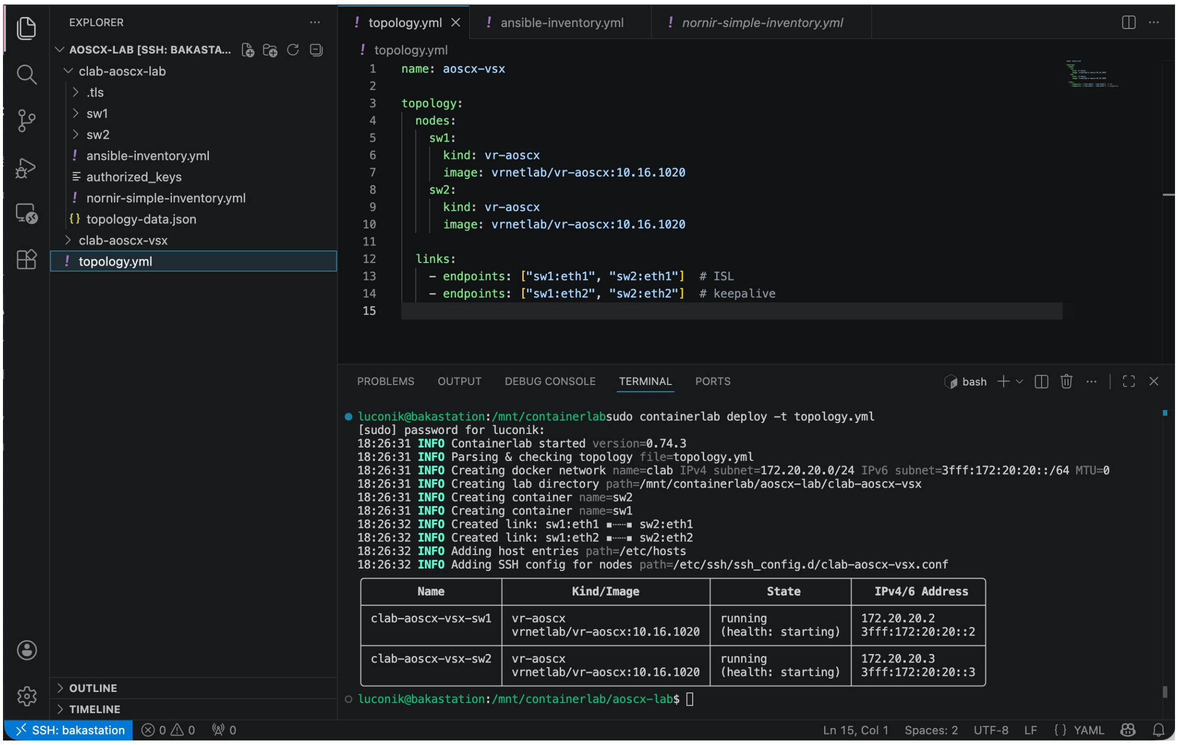
Lien / Link	Interfaces	Rôle / Role
ISL-1	sw1:eth1 ↔ sw2:eth1	Inter-Switch Link (LAG)
ISL-2	sw1:eth2 ↔ sw2:eth2	Inter-Switch Link (LAG)
Keepalive	sw1:eth3 ↔ sw2:eth3	VSX Keepalive

topology.yml

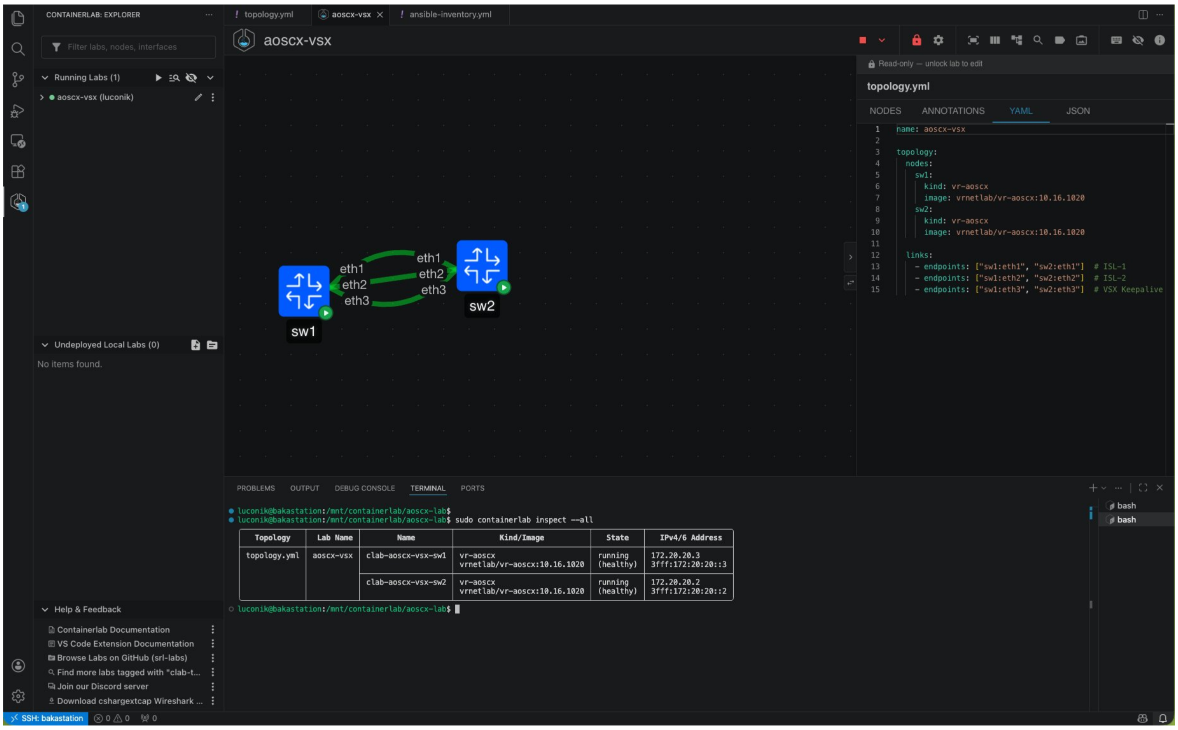
```
name: aoscx-vsx

topology:
  nodes:
    sw1:
      kind: vr-aoscx
      image: vrnetlab/vr-aoscx:10.16.1020
    sw2:
      kind: vr-aoscx
      image: vrnetlab/vr-aoscx:10.16.1020

  links:
    - endpoints: ["sw1:eth1", "sw2:eth1"]  # ISL-1
    - endpoints: ["sw1:eth2", "sw2:eth2"]  # ISL-2
    - endpoints: ["sw1:eth3", "sw2:eth3"]  # VSX Keepalive
```

Déploiement / Deployment

```
sudo containerlab deploy -t topology.yml
sudo containerlab inspect --all
# Attendre healthy (~2-3 min) / Wait for healthy state (~2-3 min)
ssh admin@172.20.20.3  # sw1 — admin/admin
ssh admin@172.20.20.2  # sw2 — admin/admin
```



Déploiement du lab AOS-CX — containers en cours de démarrage AOS-CX lab deployment — containers starting up

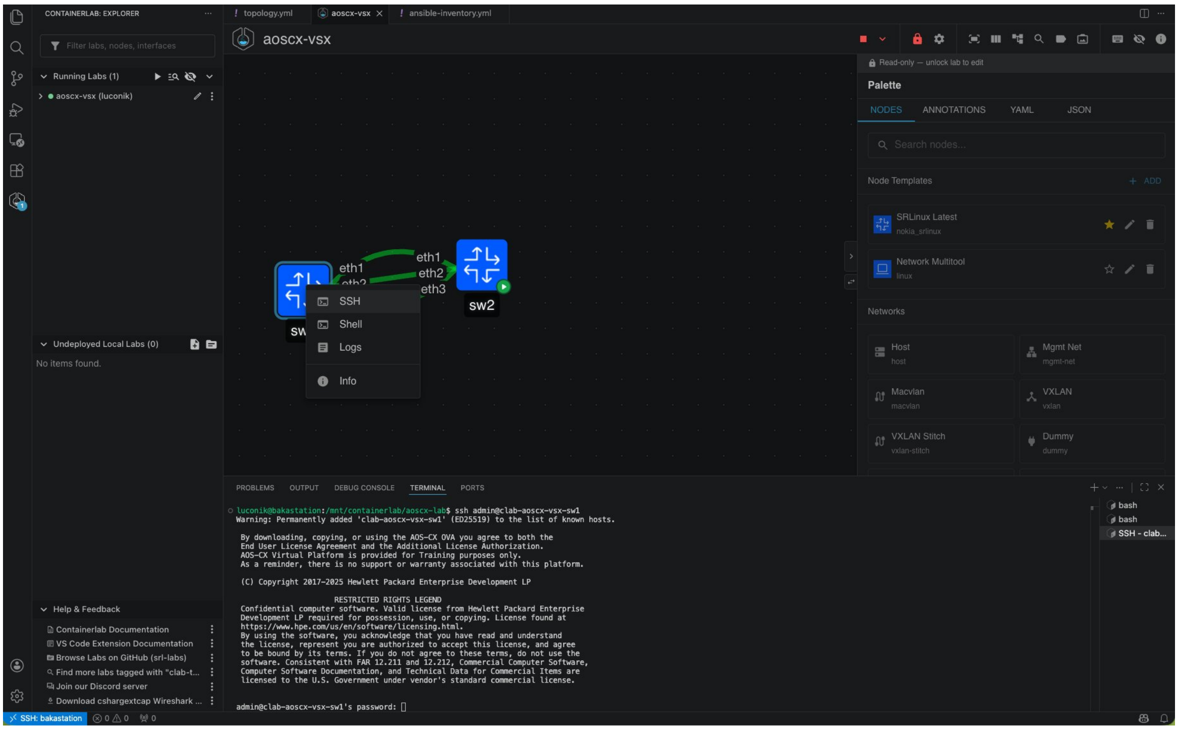


Lab déployé — topology graphique VS Code + 2 switches healthy Lab deployed — VS Code graphical topology + 2 healthy switches

Configuration VSX — SW1 (Primary)

```
configure terminal

interface lag 1
  no shutdown
  no routing
  vlan trunk allowed all
  lacp mode active

interface 1/1/1
  no shutdown
  lag 1

interface 1/1/2
  no shutdown
  lag 1

interface 1/1/3
  no shutdown
  ip address 192.168.255.0/31

vsx
  system-mac 02:01:00:00:00:01
  inter-switch-link lag 1
  role primary
  keepalive peer 192.168.255.1 source 192.168.255.0
```

■ *"no routing" doit être configuré sur le LAG avant "inter-switch-link" / "no routing" must be set on the LAG before "inter-switch-link".*



Plugin VS Code Containerlab — clic droit pour SSH direct sur un node VS Code Containerlab plugin — right-click for direct SSH on a node

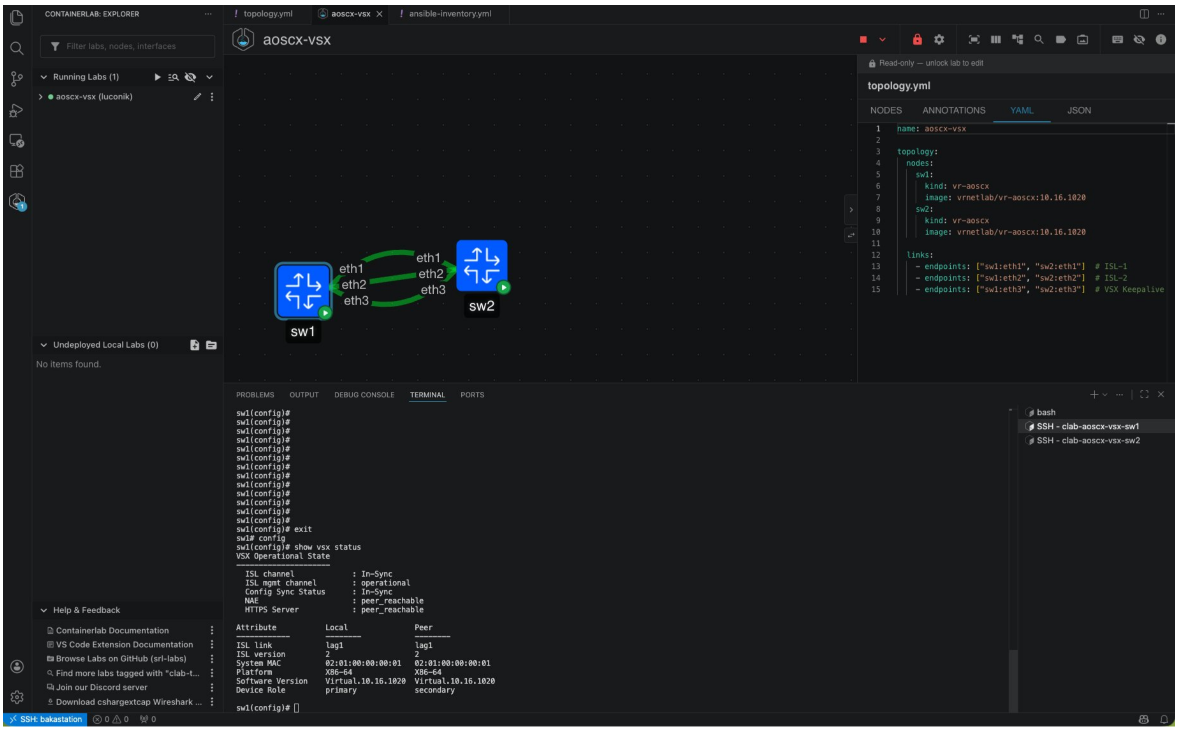
Configuration VSX — SW2 (Secondary)

```
configure terminal

interface lag 1
  no shutdown
  no routing
  vlan trunk allowed all
  lacp mode active

interface 1/1/1
  no shutdown
  lag 1

interface 1/1/2
  no shutdown
  lag 1

interface 1/1/3
  no shutdown
  ip address 192.168.255.1/31

vsx
  system-mac 02:01:00:00:00:01
  inter-switch-link lag 1
  role secondary
  keepalive peer 192.168.255.0 source 192.168.255.1
```

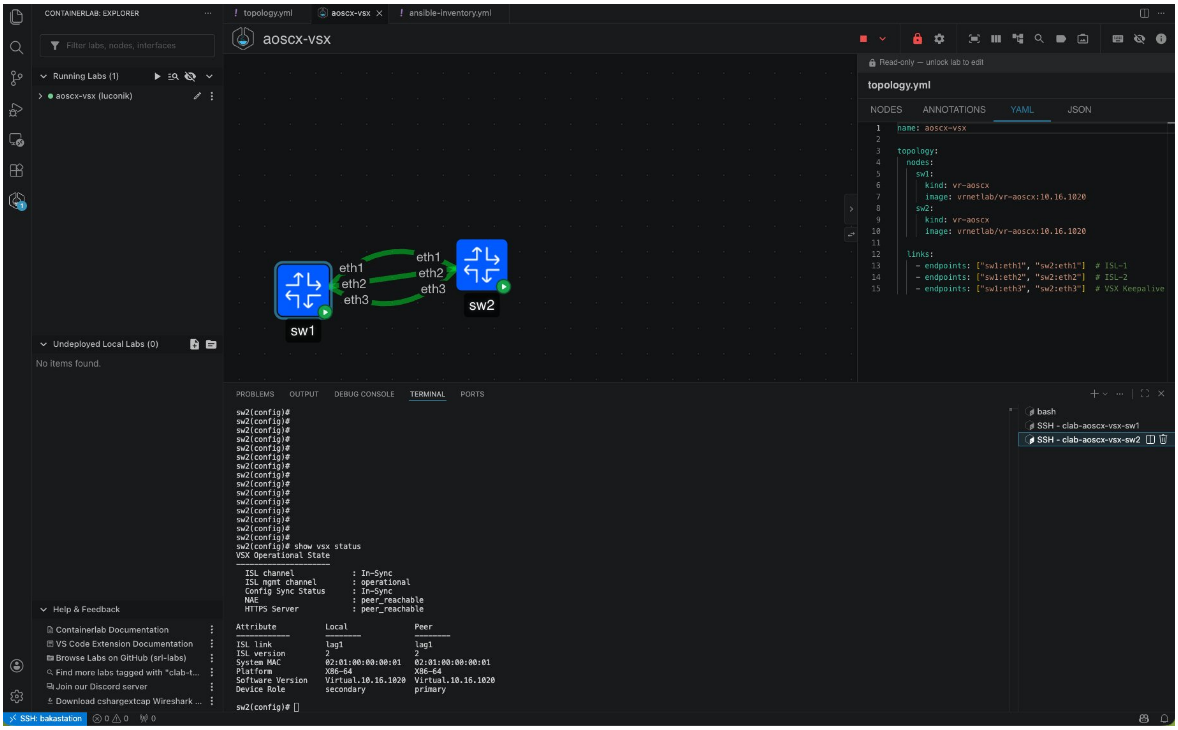
Validation VSX

```
show vsx status
show lacp interfaces
show interface lag 1
```

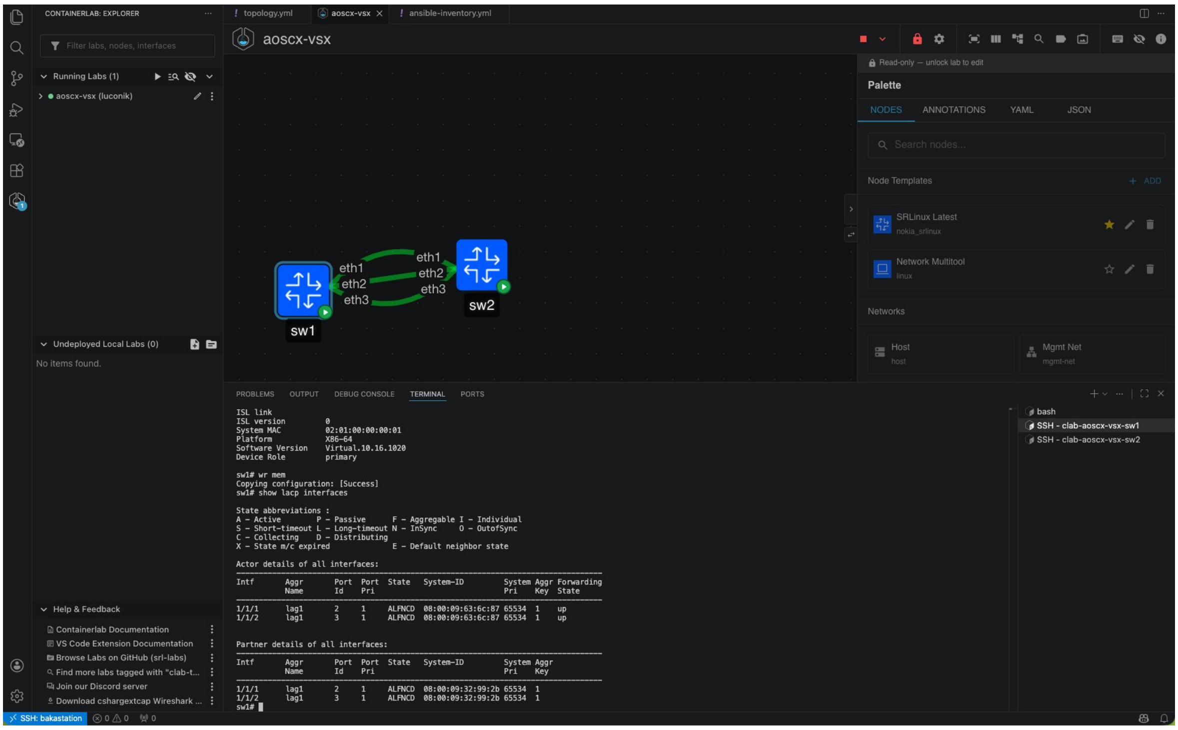
Résultat attendu / Expected result — show vsx status : ISL channel: In-Sync | ISL mgmt channel: operational | Config Sync Status: In-Sync | NAE: peer_reachable



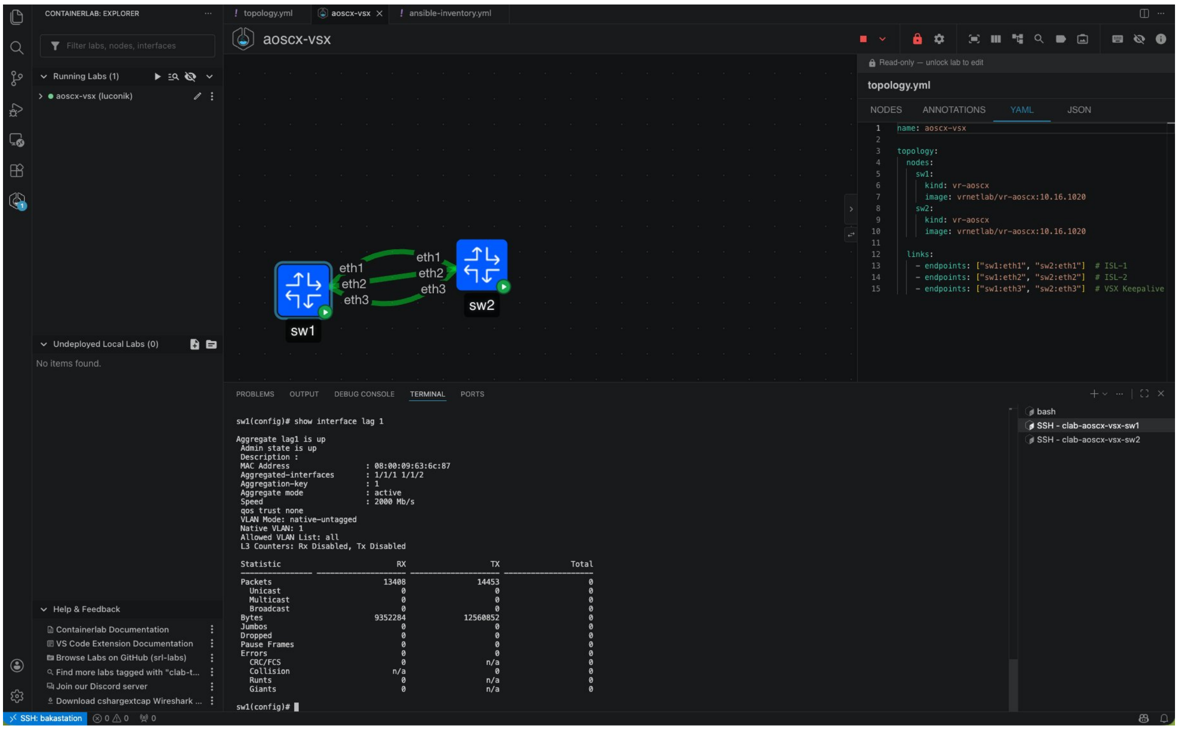
show vsx status — SW1 primary : ISL In-Sync, NAE peer_reachable



show vsx status — SW2 secondary : ISL In-Sync, NAE peer_reachable



show lacp interfaces — 1/1/1 et 1/1/2 en état ALFNCD (Active, Long-timeout, Forwarding, Collecting, Distributing)



show interface lag 1 — Aggregate lag1 up, 2x1G LACP = 2000 Mb/s

4. Lab Juniper vJunos-switch MC-LAG

■ *La configuration MC-AE complète (active/standby) nécessite du matériel physique ou EVE-NG. Ce lab couvre l'ICL LACP et l'ICCP keepalive — base de la stack MC-LAG. Full MC-AE config (active/standby) requires physical hardware or EVE-NG. This lab covers ICL LACP and ICCP keepalive — the MC-LAG stack foundation.*

Liens / Links

Lien / Link	Interfaces	Rôle / Role
ICL-1	sw1:ge-0/0/0 ↔ sw2:ge-0/0/0	Inter-Chassis Link (ae0)
ICL-2	sw1:ge-0/0/1 ↔ sw2:ge-0/0/1	Inter-Chassis Link (ae0)
ICCP	sw1:ge-0/0/2 ↔ sw2:ge-0/0/2	Keepalive 192.168.255.0/31
Client1	sw1:ge-0/0/3 ↔ client1:eth1	Ubuntu test client
Client2	sw2:ge-0/0/3 ↔ client2:eth1	Ubuntu test client

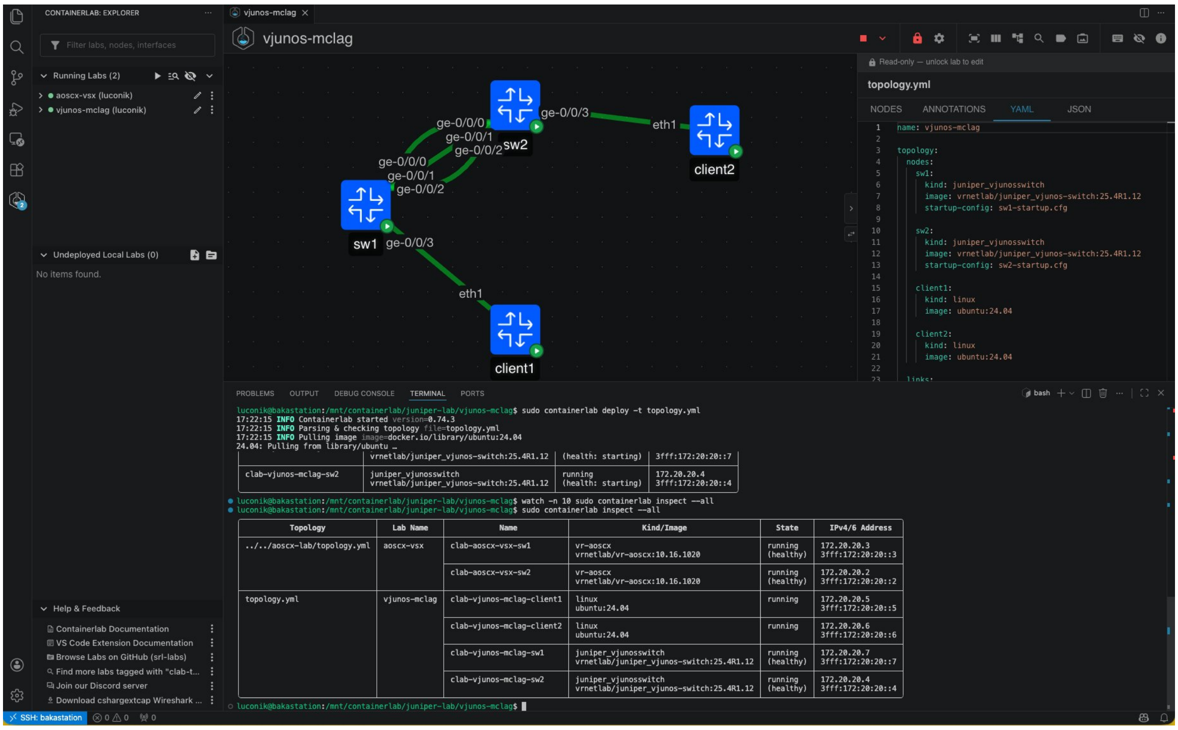
topology.yml

```
name: vjunos-mclag

topology:
  nodes:
    sw1:
      kind: juniper_vjunosswitch
      image: vrnetlab/juniper_vjunos-switch:25.4R1.12
      startup-config: sw1-startup.cfg
    sw2:
      kind: juniper_vjunosswitch
      image: vrnetlab/juniper_vjunos-switch:25.4R1.12
      startup-config: sw2-startup.cfg
    client1:
      kind: linux
      image: ubuntu:24.04
    client2:
      kind: linux
      image: ubuntu:24.04

  links:
    - endpoints: ["sw1:ge-0/0/0", "sw2:ge-0/0/0"]  # ICL-1
    - endpoints: ["sw1:ge-0/0/1", "sw2:ge-0/0/1"]  # ICL-2
    - endpoints: ["sw1:ge-0/0/2", "sw2:ge-0/0/2"]  # ICCP
    - endpoints: ["sw1:ge-0/0/3", "client1:eth1"]
    - endpoints: ["sw2:ge-0/0/3", "client2:eth1"]
```

Déploiement / Deployment

```
sudo containerlab deploy -t topology.yml
sudo containerlab inspect --all
# Attendre healthy (~10-15 min) / Wait for healthy (~10-15 min)
ssh admin@172.20.20.7  # sw1 — admin/Lab12345!
ssh admin@172.20.20.4  # sw2 — admin/Lab12345!
```

■ *Si SSH ne répond pas après "healthy" / If SSH does not respond after "healthy" : docker exec -it clab-vjunos-mclag-sw1 telnet 0 5000 login: root → configure → delete chassis auto-image-upgrade → commit*



Lab vjunos-mclag déployé — 2 switches + 2 clients Ubuntu, topology graphique VS Code vjunos-mclag lab deployed — 2 switches + 2 Ubuntu clients, VS Code graphical topology

Configuration MC-LAG — SW1

```
configure

set chassis aggregated-devices ethernet device-count 2

set interfaces ge-0/0/0 ether-options 802.3ad ae0
set interfaces ge-0/0/1 ether-options 802.3ad ae0

set interfaces ae0 aggregated-ether-options lacp active
set interfaces ae0 aggregated-ether-options lacp system-id 00:01:00:00:00:01
set interfaces ae0 aggregated-ether-options lacp admin-key 1
set interfaces ae0 unit 0 family ethernet-switching interface-mode trunk
set interfaces ae0 unit 0 family ethernet-switching vlan members all

set interfaces ge-0/0/2 unit 0 family inet address 192.168.255.0/31

set protocols iccp local-ip-addr 192.168.255.0
set protocols iccp peer 192.168.255.1 redundancy-group-id-list 1
set protocols iccp peer 192.168.255.1 liveness-detection minimum-interval 1000
set protocols iccp peer 192.168.255.1 liveness-detection multiplier 3

commit
```

Configuration MC-LAG — SW2

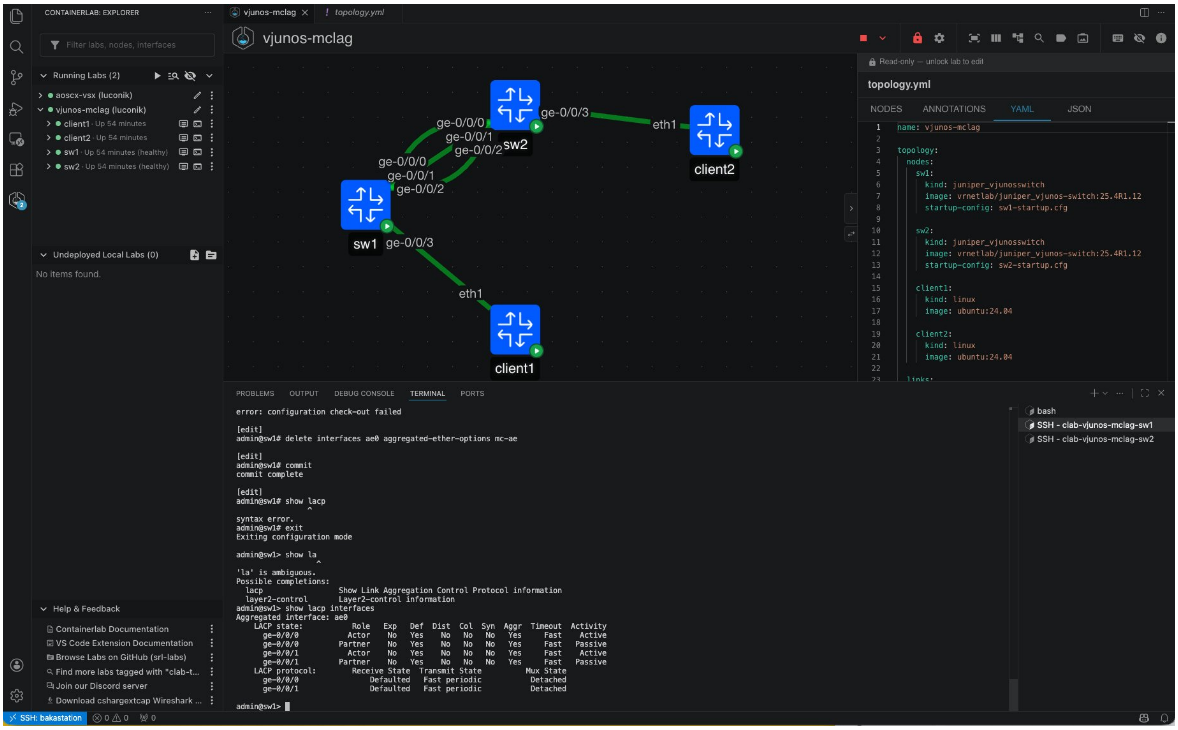
```
configure

set chassis aggregated-devices ethernet device-count 2

set interfaces ge-0/0/0 ether-options 802.3ad ae0
set interfaces ge-0/0/1 ether-options 802.3ad ae0

set interfaces ae0 aggregated-ether-options lacp active
set interfaces ae0 aggregated-ether-options lacp system-id 00:01:00:00:00:01
set interfaces ae0 aggregated-ether-options lacp admin-key 1
set interfaces ae0 unit 0 family ethernet-switching interface-mode trunk
set interfaces ae0 unit 0 family ethernet-switching vlan members all

set interfaces ge-0/0/2 unit 0 family inet address 192.168.255.1/31

set protocols iccp local-ip-addr 192.168.255.1
set protocols iccp peer 192.168.255.0 redundancy-group-id-list 1
set protocols iccp peer 192.168.255.0 liveness-detection minimum-interval 1000
set protocols iccp peer 192.168.255.0 liveness-detection multiplier 3

commit
```

■ *commit sur Junos = wr mem sur AOS-CX (config active ET persistante). "commit confirmed 5" = rollback automatique après 5 min si non confirmé — très utile en production. commit on Junos = wr mem on AOS-CX (config active AND persistent). "commit confirmed 5" = automatic rollback after 5 min if not confirmed.*

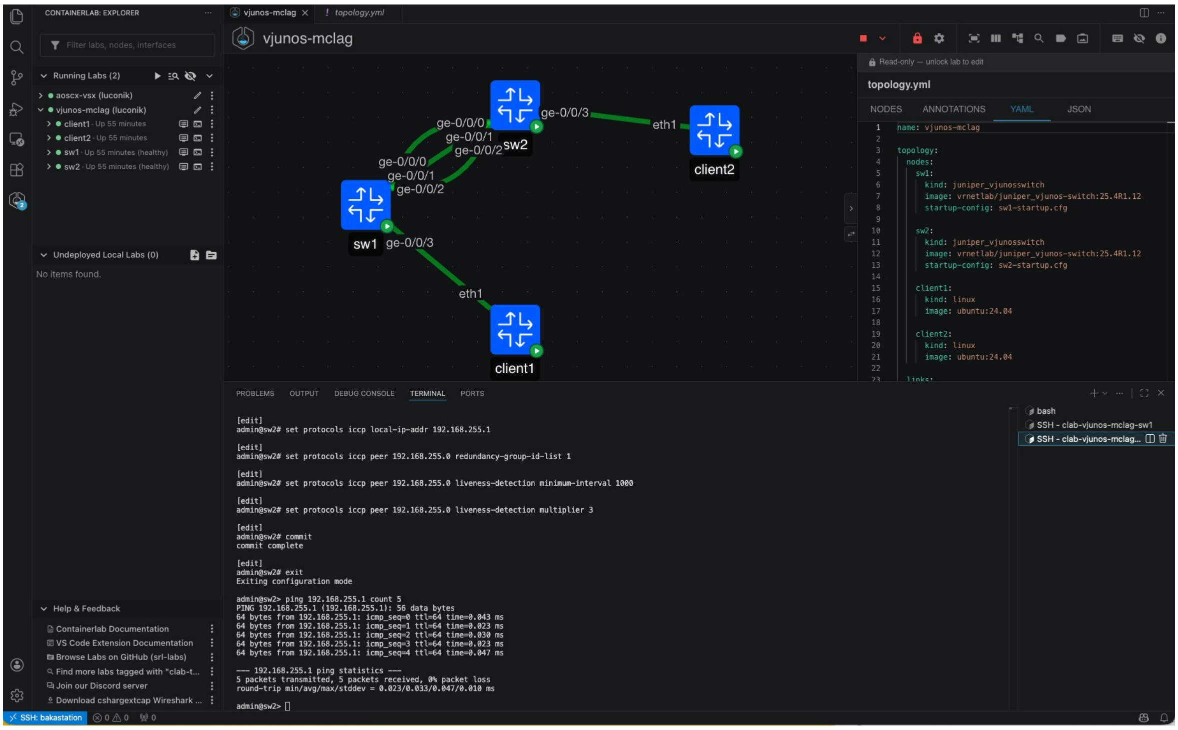
Validation MC-LAG

```
show iccp
show lacp interfaces
ping 192.168.255.1 count 5
```

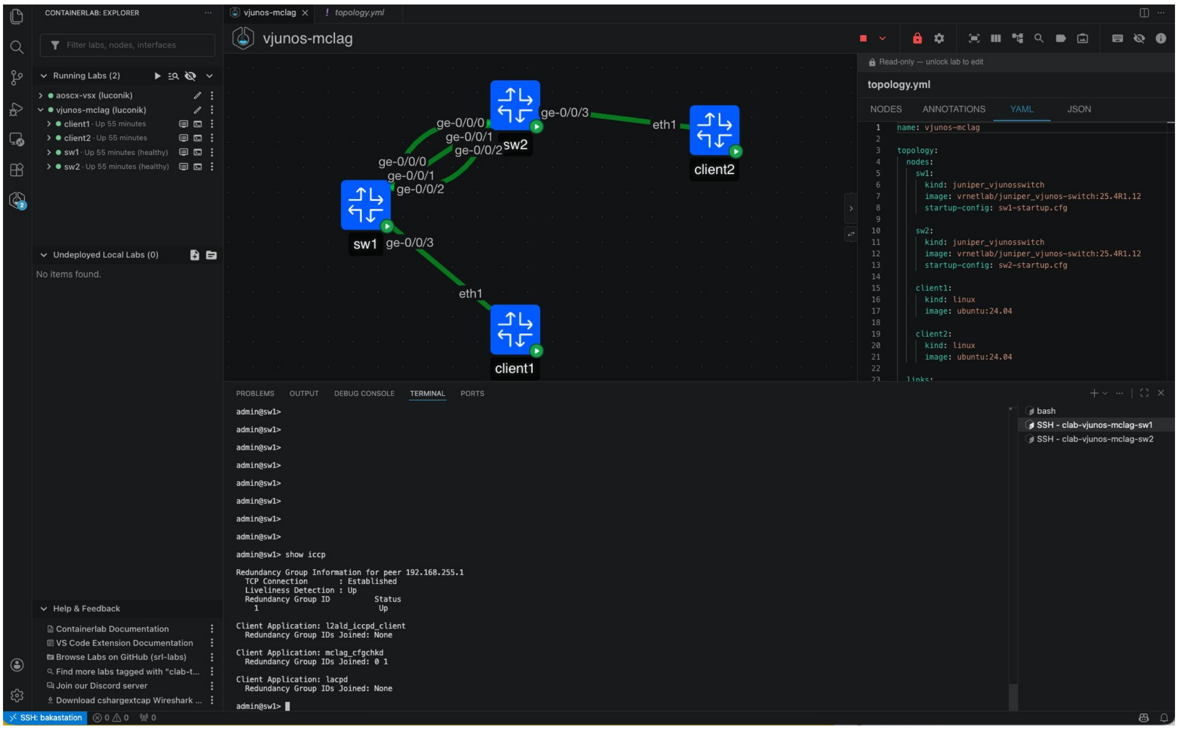
Résultat attendu / Expected result — show iccp : TCP Connection: Established | Liveness Detection: Up | Redundancy Group 1: Up



show lacp interfaces — ae0 actif avec ge-0/0/0 et ge-0/0/1 ae0 active with ge-0/0/0 and ge-0/0/1



Ping keepalive 192.168.255.0 → 192.168.255.1 — 5/5 paquets, 0% perte, ~0.033ms Keepalive ping — 5/5 packets, 0% loss, ~0.033ms RTT



show iccp — TCP Connection Established, Liveness Detection Up, Redundancy Group 1 Up

5. Comparaison VSX vs MC-LAG

Élément / Element	AOS-CX VSX	Juniper MC-LAG
Technologie	VSX	MC-LAG + ICCP
ISL/ICL	LAG LACP	ae0 (LAG LACP)
Keepalive	IP dédiée / Dedicated IP	ICCP sur ge-0/0/2
Sync protocol	VSX natif	ICCP
Rôles / Roles	primary / secondary	active / standby
Config	vsx { }	protocols iccp + mc-ae
Vérification / Check	show vsx status	show iccp
Credentials	admin/admin	admin/Lab12345!
Boot time	~2-3 min	~10-15 min

6. Plugin VS Code Containerlab

L'extension **Containerlab** (publisher : srl-labs) pour VS Code offre une interface graphique complète pour gérer les labs réseau directement depuis l'éditeur. Compatible avec **VS Code Remote SSH** — accès depuis Mac ou Windows vers Linux.

- Visualisation graphique de la topologie en temps réel
- Deploy / Destroy / Inspect sans quitter VS Code
- Accès SSH direct par clic droit sur un node (SSH, Shell, Logs, Info)
- Édition du topology.yml avec autocomplétion YAML
- Affichage simultané de plusieurs labs dans "Running Labs"

Installation

```
VS Code → Extensions → Rechercher "Containerlab" → Publisher: srl-labs → Install
```

7. Commandes de référence / Reference Commands

Containerlab — Général / General

```
# Deploy / Destroy
sudo containerlab deploy -t topology.yml
sudo containerlab destroy -t topology.yml --cleanup
sudo containerlab deploy -t topology.yml --reconfigure

# Inspection
sudo containerlab inspect --all
sudo containerlab graph -t topology.yml  # Web UI topologie

# Export configs
ssh admin@<ip> "show running-config" > sw1-config.txt
```

AOS-CX VSX

```
show vsx status
show vsx statistics
show lacp interfaces
show interface lag 1
show vsx brief
```

Juniper MC-LAG / ICCP

```
show iccp
show iccp statistics
show lacp interfaces
show interfaces ae0
show interfaces terse
ping 192.168.255.1 count 5
```

Console série vJunos / vJunos Serial Console

```
docker exec -it clab-vjunos-mclag-sw1 telnet 0 5000
# login: root (pas de mot de passe / no password)
# cli → configure → delete chassis auto-image-upgrade → commit
```